Compiler Design Laboratory Project

Phase 1 Report: Problem Definition & System Design

Student Name: Rohan R
Registration Number: 24BCE0640
Course: Compiler Design Laboratory
Project Title: NovaScript: A Lightweight Procedural Scripting Language

1. Project Title

NovaScript: Design and Implementation of a Lightweight Procedural Scripting Language and Compiler

2. Abstract

This project report outlines the Phase 1 design and problem specification for **NovaScript**, a compact, statically/dynamically scoped procedural programming language. The goal of this project is to build a complete compiler pipeline—from raw character streams down to syntax validation, symbol table management, Abstract Syntax Tree (AST) construction, and interpretation/code generation—from scratch. NovaScript supports fundamental programming constructs including variable assignments, arithmetic expressions, conditional branching (`if-elif-else`), iterative loops (`while`), and user-defined functions with parameters. By constructing NovaScript, core theoretical principles of compiler design are applied in a practical, extensible software system.

3. Problem Statement

Modern industrial compilers (such as LLVM or GCC) are monolithic and highly complex systems that obscure fundamental translation mechanisms behind extensive optimizations and target-specific layers. Students studying compiler design require a transparent, modular system that encapsulates the entire compilation lifecycle without overwhelming code volume. The problem addressed by this project is the creation of a fully functional, easy-to-medium complexity custom programming language compiler that demonstrates lexical analysis, recursive descent parsing, semantic scope checking, and AST interpretation in an educational context.

4. Motivation

Understanding how high-level human-readable code transforms into executable machine operations is essential for computer science engineering. While theoretical studies cover grammar rules and parsing algorithms, implementing a compiler from scratch bridges the gap between theory and practice. NovaScript provides an optimal balance: it is complex enough to incorporate all major compiler phases (lexing, parsing, symbol tables, type checking, semantic analysis, and code generation) while remaining compact enough for rigorous individual ownership and verification.

5. Objectives

- Design a clean, unambiguous Context-Free Grammar (CFG) for the NovaScript programming language.
- Implement a robust Lexical Analyzer (Lexer) capable of tokenizing source code and reporting lexical errors with precise line/column tracking.

- Develop a Recursive Descent Parser that transforms token streams into a structured Abstract Syntax Tree (AST).
- Construct a hierarchical Symbol Table mechanism supporting nested block scopes and variable/function lookups.
- Implement a Semantic Analyzer and Type Checker to catch declaration errors, scope violations, and type mismatches before execution.
- Provide an Execution Engine (Tree-Walk Interpreter) to run NovaScript programs accurately.
- Produce clean, well-documented code accompanied by comprehensive test cases and formal documentation.

6. Scope of the Project

The scope encompasses the full design and implementation of the NovaScript language processor. Specifically, it includes:

- **Supported Data Types:** Integers, Floats, Strings, and Booleans.
- **Control Structures:** Conditional statements (`if` , `elif` , `else`) and loops (`while`).
- **Functions:** User-defined functions supporting arguments, local scopes, and `return` statements.
- **Built-in I/O:** Native `print()` and `input()` routines.
- **Error Handling:** Structured error reporting across Lexical, Syntax, and Semantic phases.

Out of Scope: Advanced machine-code backend target generation (such as direct x86 assembly or machine binary emission) is out of scope; the project terminates at the Intermediate Representation / AST interpretation layer.

7. Background Study

Compiler construction relies on formal language theory. Lexical analysis utilizes regular expressions and Finite Automata (DFA/NFA) to group characters into tokens. Syntax analysis relies on Context-Free Grammars (CFGs) parsed via Top-Down methods (such as Recursive Descent or LL(1)) or Bottom-Up methods (LR, LALR). Symbol tables are implemented using nested hash maps or symbol environment chains to model lexical scoping rules. Semantic analysis enforces context-sensitive constraints that cannot be captured purely by context-free syntax rules.

8. Compiler Design Concepts Involved

Compiler Phase	Concept / Technique	NovaScript Implementation Details
Lexical Analysis	Tokenization, Regular Expressions	Scans source character by character; classifies keywords, operators, identifiers, and literals into tokens.
Syntax Analysis	Context-Free Grammar, Recursive Descent Parser	Parses token streams hierarchically, enforcing operator precedence and generating an Abstract Syntax Tree (AST).
Symbol Table Management	Scoped Environments, Hash Maps	Manages variable declarations, function signatures, and nested block scopes with parent pointer lookups.
Semantic Analysis	Type Checking, Declaration Validation	Verifies that identifiers are declared before use, checks function argument counts, and ensures type safety in expressions.
Intermediate Representation / Execution	AST Tree-Walk Interpretation	Evaluates AST nodes sequentially or conditionally to execute the program logic and manage runtime memory.
Error Handling	Panic-mode recovery & Diagnostic reporting	Catches lexical, syntax, and semantic errors gracefully with line number indicators.

9. Proposed Methodology

The project follows a modular, incremental development methodology split into three structured phases:

- **Phase 1 (Current):** Problem definition, system architecture design, language grammar specification, and initial prototype design.
- **Phase 2:** Core module implementation including the Lexer, Parser, Symbol Table, and Semantic Analyzer.
- **Phase 3:** Integration of the interpreter/execution engine, comprehensive testing with valid and invalid scripts, and final report compilation.

10. System Architecture

The system architecture follows a classic multi-pass compiler pipeline where data flows unidirectionally from source text to execution output:

```
[Source Code (.nova)] → [Lexer] → (Token Stream) → [Parser] → (Abstract Syntax Tree) → [Semantic Analyzer & Symbol Table] → [Tree-Walk Interpreter] → [Execution Output / Errors]
```

11. Language Specification & Grammar

11.1 Lexical Tokens

- **Keywords:** `let`, `if`, `elif`, `else`, `while`, `def`, `return`, `print`, `true`, `false`
- **Operators:** `+`, `-`, `*`, `/`, `=`, `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Delimiters:** `(`, `)`, `{`, `}`, `,`, `;`

11.2 Context-Free Grammar (BNF Subset)

```
Program    → StatementList
StatementList → Statement StatementList | ε
Statement  → LetStmt | IfStmt | WhileStmt | ReturnStmt | ExprStmt | FuncDecl
LetStmt    → "let" IDENTIFIER "=" Expression ";"
ExprStmt   → Expression ";"
IfStmt     → "if" "(" Expression ")" "{" StatementList "}" ElifList ElseBlock
WhileStmt  → "while" "(" Expression ")" "{" StatementList "}"
Expression → AssignmentExpr
AssignmentExpr → IDENTIFIER "=" AssignmentExpr | EqualityExpr
EqualityExpr   → RelationalExpr (("==" | "!=") RelationalExpr)*
RelationalExpr → AdditiveExpr (("<" | ">" | "<=" | ">=" ) AdditiveExpr)*
AdditiveExpr   → MultiplicativeExpr (("+" | "-") MultiplicativeExpr)*
MultiplicativeExpr → Primary (("*" | "/") Primary)*
Primary    → NUMBER | STRING | IDENTIFIER | "true" | "false" | "(" Expression ")" | CallExpr
```

12. Technology Stack

- **Implementation Language:** Python 3.x (chosen for expressive syntax, robust string processing, and rapid AST structure modeling).
- **Development Environment:** Visual Studio Code.
- **Version Control:** Git & GitHub repository tracking.
- **Documentation & Tools:** WeasyPrint, Python Standard Libraries.

13. Initial Prototype Design

An initial prototype of the Lexer has been structured in Python to validate tokenization logic. Below is a core snippet demonstrating how NovaScript source code is tokenized:

```
import re

TOKENS = [
    ('KEYWORD', r' (let|if|elif|else|while|def|return|print) '),
    ('NUMBER', r'\d+(\.\d+)?'),
    ('IDENTIFIER', r'[a-zA-Z_][a-zA-Z0-9_]*'),
    ('OPERATOR', r'==|!=|<=|>=|[=+\-*/<>]'),
    ('SYMBOL', r'[(){},;]'),
    ('SKIP', r'[
]+'),
    ('MISMATCH', r'.'),
]
```

```
def tokenize(code):
    tok_regex = '|'.join(f'(?P<{name}>{pattern})' for name, pattern in TOKENS)
    for mo in re.finditer(tok_regex, code):
        kind = mo.lastgroup
        value = mo.group(kind)
        if kind == 'SKIP':
            continue
        elif kind == 'MISMATCH':
            raise RuntimeError(f'Unexpected character {value!r}')
        yield (kind, value)
```